

Testing the ExecuteOrder Fix

How the race condition, idempotency, and error-reporting fixes were verified

Trading Platform

What needed proving

Replacing CheckOrder/UpdateState with a single atomic ExecuteOrder fixed three separate bugs. Each is a different *kind* of bug, so each needs a different kind of proof.

Bug	Kind of bug	Only shows up when...
Lost updates on concurrent orders (#25)	concurrency	two orders for the same user run at the same instant
Duplicate orders on retry (#13)	repetition	the same request arrives twice
Every rejection reported as “insufficient funds” (#26)	mapping	any order is rejected

The testing strategy: two layers

The tests are split into two files that do very different jobs.

Layer 1 — unit tests, with a fake engine. tests/unit/test_order_service.py replaces the risk engine with a stand-in that returns whatever answer the test wants. These are fast (milliseconds), need nothing running, and check that the *Python* side makes the right decision for each answer the engine can give.

Layer 2 — integration tests, against the real engine. tests/integration/test_execute_order_engine.py opens a real network connection to the C++ engine and sends real orders. Slower, needs the engine running, but it is the only thing that can prove the C++ behaviour.

Why both, and why the race test *has* to be layer 2

A mock is just a Python object pretending to be the engine. It has no threads, no locks, and no shared memory. The race condition lived in C++ memory guarded by a mutex — a mock cannot reproduce it, and a mock cannot prove it is gone. Testing the race with a mock would be like testing whether a door lock works by drawing a picture of a door.

Equally, running everything through the real engine would be slow and would mean you cannot run the test suite without first starting a C++ process. So the split is deliberate: **test the decision logic with mocks, test the concurrency guarantees for real.**

The integration file skips itself automatically if nothing is listening on port 50051, so pytest still works on a machine with no engine running.

Layer 1: the unit tests

File: tests/unit/test_order_service.py — 8 tests.

How they work. Each test builds an OrderService whose risk engine is a MagicMock programmed to return one canned response — say, “result = INSUFFICIENT_FUNDS, required_cash = 500, available_cash = 100”. The test then calls place_order and checks what comes out.

Why mock instead of using the real engine. We are not testing whether the engine does the right arithmetic here. We are testing whether Python, *given* an answer, turns it into the right HTTP outcome. Mocking makes each test one line of setup, removes the need for a database or a network, and lets us produce answers that would be awkward to trigger for real.

Test	What it checks
test_filled_returns_order_built_from_engine_response	On a fill, Order.id is the engine’s id (not a UUID Python invented) and idempotency_key is the client’s id
test_filled_passes_client_order_id_through_unchanged	The client’s id reaches the engine untouched — if Python rewrote it, retries would never be recognised
test_unknown_symbol_raises_symbol_not_found	UNKNOWN_SYMBOL → 404
test_limit_not_met_has_its_own_error	LIMIT_NOT_MET → its own 422 error, not “insufficient funds”
test_insufficient_funds_reports_the_engine_s_numbers	The error message contains the real shortfall (500) and real balance (100)
test_insufficient_position_reports_the_engine_s_numbers	Same, for selling more than you hold
test_idempotency_conflict_raises_duplicate_request	IDEMPOTENCY_CONFLICT → 409
test_invalid_order_raises_invalid_order	INVALID_ORDER → 400

The last five exist because of a specific old bug: CheckOrder returned a free-text reason string that Python never read, so it guessed — BUY rejections became “insufficient funds”, SELL rejections became “insufficient position”, always. A limit order priced away from the market was reported as being out of money. These tests make that impossible to reintroduce.

Layer 2: the integration tests

File: tests/integration/test_execute_order_engine.py — 10 tests.

Making the tests deterministic

Two problems had to be solved before any assertion could be reliable.

Problem 1: the price keeps moving. The engine prices orders from the live Binance order book. A test that says “buying 1 unit should cost exactly X” would fail seconds later when the market moved.

Solution: the test creates its own symbol. UpdateOrderBook is a normal RPC — it is how the Binance feed pushes prices into the engine. The test uses that same RPC to push in a made-up symbol, TESTUSDT, with the bid and the ask both set to exactly 100.00. Now every number in the test is predictable: buying 1 unit costs exactly 100, every time.

Problem 2: accounts do not reset. LoadUser deliberately does nothing if the engine already knows that user — you would not want someone logging in again to reset their balance. So a test that used a fixed user id would find

a drained account on its second run.

Solution: a random user id per run, drawn from 10,000,000–99,999,999 so it cannot collide with a real account.

The result is that these tests need **no database rows, no fixtures, and no cleanup**. Each one builds the small world it needs and leaves nothing behind that a later run depends on.

The race test — the important one

`test_concurrent_orders_for_one_user_do_not Lose_updates`

What it does. Starts with a user holding 10,000 cash. Fires **20 BUY orders at the same time**, from 20 threads, all for the same user, each buying 1 unit at 100.

What it asserts, and why each assertion matters:

1. **All 20 come back FILLED.** The user can afford 100 units; nothing should be rejected.
2. **The sequence numbers are exactly 1, 2, 3 ... 20.** Sorting the sequence numbers from all 20 responses must give precisely that list. This single check catches two different failures: a *repeat* (two orders both claiming to be number 5, meaning one overwrote the other) and a *gap* (a number missing, meaning a commit vanished).
3. **The final balance is 8,000** — that is 10,000 minus 20×100 . Every single deduction is accounted for.
4. **The final position is 20 units.**

What would have happened before the fix. With `CheckOrder/UpdateState`, two concurrent orders could interleave like this:

```
Order A: CheckOrder  -> reserves, cash 10,000 -> 9,900, releases lock
Order B: CheckOrder  -> reserves, cash  9,900 -> 9,800, releases lock
Order A: UpdateState -> writes back its snapshot: cash = 9,900    <-- B erased
Order B: UpdateState -> writes back its snapshot: cash = 9,800
```

Depending on the timing, one order's deduction is silently overwritten by the other's stale snapshot. The user ends up with units they did not pay for. With 20 orders racing, the loss compounds.

Why 20 orders and not 2. A race condition is a timing window. With two orders you might get lucky and never land inside it, so a broken build would pass most of the time — the worst kind of test. Twenty concurrent orders on a thread pool makes the interleaving overwhelmingly likely, so a regression fails loudly rather than occasionally.

The idempotency tests

`test_duplicate_client_order_id_executes_only_once`

Sends the exact same order twice, with the same `client_order_id`. Both come back FILLED — a retry should not look like an error to the caller — but the test then checks **four** things are identical between the two responses:

- the same `order_id`
- the same `event_id`
- the same `account_sequence`
- the same resulting cash

and that the cash is 9,900, not 9,800. **Charged once, not twice.**

Checking all four matters because each rules out a different way of getting it wrong. Matching cash alone could in principle happen by accident; a matching `event_id` proves no second commit event was ever created, and a matching `account_sequence` proves the per-user counter never ticked a second time.

`test_reusing_a_key_for_a_different_order_conflicts`

Sends `client_order_id = "conflict-key"` for 1 unit, then the same key for 2 units. Expects `IDEMPOTENCY_CONFLICT`.

This proves the engine is not naively handing back whatever is in its cache. It actually compares the incoming order against the stored one, and when they disagree it refuses rather than guessing which order the caller meant.

The rejection tests

Test	Proves
<code>test_unknown_symbol_is_reported_as_such</code>	A symbol the engine has never seen returns <code>UNKNOWN_SYMBOL</code>
<code>test_insufficient_funds_reports_the_real_shortfall</code>	Ordering 1,000 units (100,000 needed, 10,000 available) returns those actual figures — the old code reported “requires 0”
<code>test_limit_order_that_does_not_cross_is_rejected_distinctly</code>	A BUY limit at 90 against a market at 100 returns <code>LIMIT_NOT_MET</code> , its own code
<code>test_engine_rejects_malformed_quantities</code>	Four bad quantities — 0, -1, NaN, infinity — each return <code>INVALID_ORDER</code>

Why the malformed-quantity test is written as one test with four inputs. It is *parametrized*: the same test body runs four times with different values, and each run is reported separately. If NaN is handled but infinity slips through, the output names exactly which one failed, instead of one lumped failure.

Why NaN and infinity specifically. They are the cases that look handled but usually are not. The engine’s check is written as:

```
!(request->quantity() > 0.0)
```

Any comparison involving NaN is false, so `NaN > 0` is false, so `!false` is true — rejected. The obvious-looking alternative, `quantity <= 0`, would **let NaN straight through**, because `NaN <= 0` is also false. That is a one-character difference between a correct check and a hole, which is exactly the kind of thing a test should pin down.

Why validate in the engine at all, when the API already checks quantity? Because the engine is not only reached through the API. The strategy runner calls it, and any future service would too. Validating at the boundary that actually mutates state means a bug in one caller cannot corrupt an account.

Running the tests

Everything that needs no engine (23 tests — the unit tests plus the existing route tests):

```
poetry run pytest -q
```

The 10 engine tests will show as skipped.

The engine tests. In one terminal (MSYS2 UCRT64, from the repo root):

```
./risk_engine_cpp/build/risk_engine.exe
```

Wait for gRPC server listening on port 50051. Then in a second terminal:

```
poetry run pytest tests/integration/test_execute_order_engine.py -v
```

Postgres must be up (`docker compose up -d`) because the engine reads `market_prices` on startup — but the tests themselves touch no tables.

Current result: 33 passing — 23 without the engine, 10 more with it.

What is deliberately not covered yet

Being honest about the edges matters more than claiming full coverage.

- **Crash recovery mid-fill.** If the engine dies between mutating memory and the WAL flush, that fill is lost. Not tested, and not currently handled.
- **Kafka reordering across partitions.** `account_sequence` is written into every event so a consumer *could* detect out-of-order delivery, but nothing acts on it yet. The topic has one partition today, so ordering holds for free (GitHub #24).
- **The engine losing its idempotency cache on restart.** The database's unique constraint on `orders.idempotency_key` is the backstop, but there is no test that restarts the engine mid-scenario.
- **Decimal precision and rounding.** All money is currently `double`. Rounding behaviour at the boundaries is untested, pending the currency/precision work.
- **Orders for *different* users running in parallel.** They should be fully concurrent, since each user has their own lock — but no test asserts that they actually are.